# The Goal

Design and implement a simple LRU caching mechanism for file blocks. It is a response to a fun little challenge I received a while ago.

# Disclaimers

This document is HAND-WRITTEN, not using AI. All the typos are the result of awesome creativity of my brain-hand nexus. For AI-Assisted version with spell-checks and proof-reading, please refer to README-AI.md.

Please refer to the LICENSE.md for details on the licensing terms.

This is original work, built on various amazing bits of software by some incredibly smart people. All credits to them where due. Please refer to the individual licenses of each software subsystem/library used for individual detailed licensing terms.

This document and accompanied work is provided as is, with any guarantees explicit or otherwise. Consider this work is unlicensed where software warranties and/or guarantees are mandated by the law or otherwise. Use at your own risk, please.

Finally, this is just a demonstration of the concept. Kindly note, there may be better and alternative implementations. Weigh in your own CBA before considering using this work.

# Sub Goals

1. Keep it light-weight and simple.
2. Keep it as generic as possible.
3. Make it easy to read, even for an intern and/or student.

# Context

Filesystems and document databases operate at the file level context. While operating at file level, applications may benefit huge from caching specific contents of file(s) they work with.

# Considerations

## Direct IO and General Use case

The caching provided by the filesystem and the underlying block layers usually is sufficient for most applications. If an application prefers to use its own caching, it is preferable to use the Direct IO to bypass these caching layers. This is a separate topic, demanding its own discussion and considerations.

The example usecases for such application level caching is database files, deduplication engines for backup etc, where application benefits greatly from its private caching mechanisms over shared OS level caching.

## Caching Algorithms and Models

In general, LRU Caching may not always be the optimal model for most applications, especially with smaller cache capacities. Other models exist such as clock-tracking or usage count tracking etc. for consideration.

Hybrid models such as LRU, weighted with age could sometimes offer better cache-hit performance.

This code, of course, could be easily adapted to accommodate such hybrid models.

## Cache Tagging

Since we are operating at the file level context, the cache tagging is based on some combination of FBN. For a global and lasting context, using the additional Inode number helps last the cache tags across sessions. Alternatively, transient caches can be managed by simple combinations of FBN and FD.

Another alternative is to use the LBA of the block. This, however, requires the additional use of volume/partition reference such

as UUID to keep the tag globally unique.

### Fetching the FBN

It is possible to directly read the FBN by opening, parsing and traversing the inode(s). The additional complexity is worth at scale. But this is skipped for now in favour of Deriving the FBN.

### Deriving the FBN

The figurate analog of the FBN can be calculated from the file offset using simple math: (offset / FILE_BLOCK_SIZE). Typically, it is 4KB. Note that the FILE_BLOCK_SIZE may be fine-tuned dependent on quite a few factors such as the Filesystem, OS/Kernel version, block storage layers below etc. While the choices for personal computing equipment typically ranges from 1KB to 4KB, the large file blocks are not that common. For large file servers, it could be fine-tuned to larger blocks, for example, to 1+MB. The merits and compromises of file block sizes are beyond the scope of this document.

If uniformity is the goal, the FILE_BLOCK_SIZE could be specified manually. However, this adds little benefits even in the systems where multiple filesystems provide multiple block sizes.

### Other ways of Tagging

Application may provide it's own way tagging the cached blocks. This, for example, could use UUID generators, internal references etc. Since this is specific to the targeted application, and involves understanding the internals of the target application, this is kept aside for now.

### Sparse Files

Sparse files may provide a challenge for file-block type of caching as they may fill up cache lines with empty blocks. This is especially true when scanning the contents of a document file, for example. It is left as an enhancement to add parse-file handling to this caching model.

## Design Considerations

Filesystem caching is hard, complex and extremely difficult to maintain consistency. This implementation is intentionally chosen to not provide production quality code.

There are a few design considerations: Cache Line Look Up, LRU Chain Management, Orphan Control, Session Management.

The Orphan Control to scan and manage orphan blocks is left out of the current scope.

Session Management to save/restore session etc. are left out of the current scope, too.

## Cache Line Look Up

## Design Choices

Three choices are considered: Doubly-linked Lists, Hash Tables and Binary Trees.

## Doubly-linked Lists

This is the simplest implementation. Each new block is appended to the end of the list. Look up is done by scanning the list until the Cache Tag is found and it is returned.

The advantage of this model is it's simplicity of implementation and ease of testing.

The primary disadvantage is, it is a linear algorithm - O(n). If the LRU Chain grows by orders of magnitude, the time-to-look-up may start to suffer. Given the probability of accessing the recently used block again soon, this might actually start hurting the performance of the primary data path more.

This can be mitigated to an extent by flipping the list. Inserts put the Cached Block at the beginning of the line. This speeds up the look up of recently cached blocks.

There are still other considerations. For example, this model destroys cache-lines, triggers a lot of random reads from multiple RAM banks specifically when traversing the list. This is especially true for real-world scenarios with thousands and millions of cache blocks.

In other words, this is not a scaleable design. Not suitable choice beyond a few hundred cached blocks.

# Hash Tables

Computationally, this is the best solution - O(1). Constant time and computational complexity. Almost constant memory complexity. In fact most Set-Associative Caches use this approach.

The only issue with this approach is, we need memory - a lot of it. Depending on the hash entry size, for example, a 32-bit hash table needs just about 16GB of memory.

The other issue is with collisions. With consideration for the Pigeon Hole Principle, the number of blocks yielding the same hash would be more than 1.

These issues can be mitigated, of course. For example, the memory requirement can be managed using a memory mapped sparse file. This mitigates the need of storing the entire hash table in memory, at the cost of initial loading time.

For the collisions, the blocks could be managed using the doubly-linked list.

These and other such mitigations do come at some form of costs. As it can be seen from the theory, this is still a far superior solution to the doubly-linked list solution.

# Binary Trees

Cache tags are organized in a binary tree, organized by deriving from tag. If the tag is single word-boundary, such as a DWORD or QWORD, it could be used as it is. The larger tags may need some sort of hash functions to derive the node ID for better performance. Hash collisions can be managed at the leaf-nodes by using the actual tags as further

Typically, self-balancing trees such as Red Black Trees provide best balance between cyclomatic and space complexities.

The standard library provides two implementations ready to use: std::set and std::map.

This approach is selected for Cache Line Look Up.

# LRU Chain Management

Multiple design options exist for LRU Chain Management, too.

The doubly-linked list with append could be a choice. In fact, the look up list could be reused as it is. However, this still carries all the benefits and issues from the look up.

As an alternate, a sparse list of tag IDs could be maintained in RAM, such as a hash table.

Another approach is to organize the tag IDs into another self-balancing binary tree with the timestamp as the key. This, however, needs an additional requirement of tracking the last used time in the Look-up datastructure, and balancing the tree every time a block is accessed.

Hence, the choice is between two O(log n) computational complexity and linear memory complexity.

In real world production implementations, it is usually a hybrid approach with the last accessed tag IDs maintained in a vector/ array and the LRU tree is only synchronized periodically. Thus, any blocks looked up from the Cache Line Tree could be ignored if the tag ID exists in the recently accessed tag IDs list.

# Locks and Race Conditions

This is a rather complex and deep area.

Locks or lockless algorithms are needed against both the trees or the linked list(s) for maintaining consistency and synchronization.

This is left out of scope for the current implementation.